

SL2DT-07: User Governance & Access

Series: SL2DT | **Notebook:** 7 of 10 | **Created:** April 2026 | **Last Updated:** 04/21/2026

Overview

Goal of this step: translate Sumo's RBAC model (roles + search filters + capabilities) into Dynatrace Platform IAM (groups + policies + bucket scoping). Do not defer this work — IAM decisions are made in Wave 1 of the migration, not at the end.

Governance drift — users without proper policies, buckets without proper scoping, SSO not mapped — causes cutover delays more often than query translation issues. Get this right before Wave 2.

Table of Contents

1. [What You'll Produce](#)
 2. [Sumo → Dynatrace IAM Mapping Model](#)
 3. [Bucket-Scoped Policies — Implementation](#)
 4. [SSO & User Provisioning](#)
 5. [Role-Based Dashboard/Monitor Access](#)
 6. [Audit Trail Parity](#)
 7. [Governance Pitfalls to Avoid](#)
 8. [Step Exit Criteria](#)
-

Prerequisites

Requirement	Details
Audience	Platform admins + security team
Inputs	<code>inventory/roles.json</code> , <code>users.json</code> , <code>rbac-mapping.md</code> from SL2DT-02
Dynatrace access	Platform Token with <code>iam:groups:write</code> , <code>iam:policies:write</code> , <code>settings:objects:write</code>
SSO context	Existing IdP details (Okta, Entra, Ping)
Prior reading	IAM-01 (Platform IAM fundamentals), IAM-07 (bucket policies)

1. What You'll Produce

Artifact	Purpose
<code>iam/groups.tf</code>	Terraform for IAM groups (one per Sumo role)
<code>iam/policies.tf</code>	Policies (scoped by bucket)
<code>iam/bindings.tf</code>	Group→policy bindings
<code>iam/sso-mapping.md</code>	IdP claim → group membership rule
<code>iam-rollout-plan.md</code>	Sequence: shadow-mode → cut-over → retire-legacy

2. Sumo → Dynatrace IAM Mapping Model

The Three Layers

Sumo Role		Dynatrace IAM Model
Role name	→	Group name
Capabilities	→	Policy permissions
Search filter	→	Policy bucket/scope filter
User assignment	→	Group membership (often via SSO claim)

Example Mapping

Sumo role: prod-readers

- Search filter: `_sourceCategory=prod/*`
- Capabilities: Dashboard view, Search, Monitor view (no edit)
- Assigned users: 82

Dynatrace equivalent:

```
# Group
resource "dynatrace_iam_group" "g_prod_readers" {
  name = "g_prod_readers"
  description = "Read-only access to production logs + dashboards"
}

# Policy (bucket-scoped)
resource "dynatrace_iam_policy" "p_prod_readers" {
  name = "p_prod_readers"
  environment = var.tenant_uuid
  statement_query = <&lt;&lt;-EOT
    ALLOW storage:logs:read, storage:events:read, storage:metrics:read
    WHERE storage:bucket-name = "custom_logs_prod";
    ALLOW document:documents:read;
  EOT
}

# Binding
resource "dynatrace_iam_policy_bindings" "b_prod_readers" {
  group = dynatrace_iam_group.g_prod_readers.id
  environment = var.tenant_uuid
  policy = [dynatrace_iam_policy.p_prod_readers.id]
}
```

Policy Language

Dynatrace policies use a declarative grant syntax:

```
ALLOW <permission>; <permission>; ...
WHERE <condition>;
```

Common bucket-scope conditions:

- `storage:bucket-name = "bucket_x"`
- `storage:bucket-name IN ("a", "b", "c")`

- `storage:table = "logs"`
- `document:owner-id = "$subject"` (user owns the document)

3. Bucket-Scoped Policies — Implementation

Bucket scoping is the primary isolation mechanism. Apply it for every team/environment boundary.

Policy Patterns

Pattern 1 — Read-only to one bucket

```
ALLOW storage:logs:read, storage:events:read
WHERE storage:bucket-name = "custom_logs_prod";
```

Pattern 2 — Read all, write to own bucket

```
ALLOW storage:logs:read
WHERE storage:table = "logs";
ALLOW storage:logs:write
WHERE storage:bucket-name IN ("custom_logs_team_payments");
```

Pattern 3 — Admin on multiple buckets

```
ALLOW storage:logs:read, storage:logs:write, storage:events:read
WHERE storage:bucket-name LIKE "custom_logs_prod%";
ALLOW document:documents:read, document:documents:write;
ALLOW settings:objects:read, settings:objects:write
WHERE settings:schema-id = "builtin:monitoring.slo";
```

Pattern 4 — Full admin (limit to platform engineering)

```
ALLOW *;
```

Avoid

- `ALLOW *` on team-level groups. Escalation risk.

- Mixing wildcards and specific grants in the same policy — harder to audit.
- Permissions without WHERE clauses — silently grants tenant-wide access.

Validate Policies

After applying, verify a test user in the group can (and cannot) do what the policy says:

```
// Verify group members' effective policies
fetch dt.iam.policies, from:-5m
| filter name startsWith "p_prod"
| fields name, statementQuery
| sort name asc
```

4. SSO & User Provisioning

SCIM / SAML Integration

Most migrations preserve the same IdP (Okta / Entra / Ping). Map IdP groups to Dynatrace groups via SAML assertion attribute or SCIM group push.

Okta example — SAML assertion:

```
dynatrace.group = "g_prod_readers,g_payments_team"
```

Dynatrace parses the `dynatrace.group` attribute on login and assigns memberships.

SCIM push (preferred for lifecycle management):

- Okta SCIM connector writes group membership on user provisioning
- Membership changes propagate automatically
- Deprovisioning is immediate

User Lifecycle

Event	Sumo Behavior	Dynatrace Behavior
New hire	Manual add in Sumo admin	SCIM push → auto-assigned to groups
Team change	Role manually reassigned	SCIM push reassigns group memberships

Event	Sumo Behavior	Dynatrace Behavior
Termination	Manual disable	SCIM push removes all memberships

Provisioning Tracker

```
| User | Sumo Role | Target DT Group | SSO Push Confirmed |
|-----|-----|-----|-----|
| jane@example.com | prod-readers | g_prod_readers |  2026-05-01 |
| ... |
```

5. Role-Based Dashboard/Monitor Access

Dashboards, notebooks, and workflows have their own access model (via `document:documents:*` permissions) separate from bucket scoping.

Document Permission Levels

Permission	Effect
<code>document:documents:read</code>	Can view own + shared
<code>document:documents:write</code>	Can edit own + shared
<code>document:documents:read WHERE document:owner-id = "\$subject"</code>	Own only
<code>document:documents:read WHERE document:shared-with = "\$subject"</code>	Shared with me
<code>document:documents:admin</code>	Can modify any document (platform admins only)

Default Shares

Set dashboard defaults during migration so new dashboards are readable by the team:

- Every team dashboard: shared with the team's IAM group (read)
- Every team notebook (operational): shared with team (read)
- Every team workflow: owned by team admin group (read/write)

Content Folders

Dynatrace Documents have a flat namespace with tags/shares, not folder hierarchy like Sumo. Map Sumo folders to tags:

Sumo Folder	Dynatrace Tag
Payments/Prod	team:payments , env:prod
Identity/Shared	team:identity
Archive	status:archived

6. Audit Trail Parity

Sumo's audit log captures every user action. Dynatrace has equivalent via `audit` events in Grail.

Sumo audit categories → Dynatrace equivalents

Sumo	Dynatrace
Login / Logout	<code>event.category = "authentication"</code>
DashboardViewed / DashboardEdited	<code>event.category = "document"</code> with action
MonitorCreated / MonitorUpdated	<code>event.category = "settings"</code>
Search executed	<code>event.category = "storage.query"</code>
User/Role changes	<code>event.category = "iam"</code>

Query audit events

```
// Dynatrace audit events
fetch events, from:-24h
| filter event.category == "iam"
| summarize c = count(), by:{event.action, user.name}
| sort c desc
```

Retention

Audit data goes in the `audit_logs` bucket (from SL2DT-03). Retention must match compliance requirements (usually 365d+). Verify in the bucket config:

```
resource "dynatrace_platform_bucket" "audit_logs" {  
  name      = "audit_logs"  
  retention = 365  
}
```

7. Governance Pitfalls to Avoid

Pitfall 1 — "We'll do IAM at the end"

Deferring IAM means rebuilding ingest (bucket decisions are coupled to policy scope). Do IAM in Wave 1.

Pitfall 2 — Overly broad initial groups

Tempting: start with one group `all-users` with `ALLOW *`. Migration pressure compounds: "we'll tighten later." Later never comes, and when it does, revoking access breaks people's workflows.

Fix: start with the Sumo role structure as-is (one DT group per Sumo role, same scope). Refactor later with data.

Pitfall 3 — Unscoped dashboard writes

A dashboard created by a team member ends up without share settings, then becomes unreachable when they leave. Every dashboard should have a group owner, not a user owner.

Fix: enforce via PR review during SL2DT-06. Check the default-share policy.

Pitfall 4 — SSO group-name mismatch

Okta pushes group name `payments-platform-prod` ; Dynatrace group is `g_prod_readers` . Users log in but get no permissions.

Fix: document SSO-claim → DT-group mapping in `iam/sso-mapping.md` . Test with at least one user per group before cutover.

Pitfall 5 — Missing audit events for bucket-level access

Bucket read/write permissions are evaluated per-query. Audit events must include `storage:bucket-name` to trace who queried what.

Fix: verify audit retention + query patterns in the audit bucket. See OPLOGS-09 for audit-query patterns.

8. Step Exit Criteria

G7 — Governance Ready

- [] IAM groups created (one per Sumo role)
- [] Policies created with bucket scoping
- [] Group↔policy bindings applied
- [] SSO mapping documented and tested with at least one user per group
- [] Dashboard/notebook share defaults configured
- [] Audit events flowing into `audit_logs` bucket with ≥365d retention
- [] Rollout plan documented (shadow-mode → cutover → retire legacy)

Next step: SL2DT-08 — Automation & GitOps (Monaco/Terraform for config promotion, CI/CD integration).

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official [Dynatrace documentation](#) and [Sumo Logic documentation](#).